

FASE 3

Monitoramento Contínuo IoT na Saúde

Projeto CardioloA — Integração de IoT, computação em nuvem, Edge/Fog Computing e visualização de dados.

Sobre a atividade

Nesta fase, você e sua equipe irão avançar no projeto CardioloA, integrando conceitos de IoT, computação em nuvem, Edge/Fog Computing e visualização de dados. O desafio é simular um sistema de monitoramento contínuo que capture sinais vitais de pacientes cardiologistas, armazene e processe informações localmente, transmita os dados para a nuvem e exiba resultados em dashboards interativos.

Você irá trabalhar com ESP32 no Wokwi, protocolos de comunicação (MQTT), armazenamento local (SPIFFS) e dashboards (Node-RED/Grafana). Essa combinação permitirá compreender o fluxo completo de dados em IoT aplicado à saúde: captura → processamento → transmissão → visualização → alerta.

Objetivo geral

Desenvolver um protótipo funcional que simule um sistema vestível de monitoramento cardíaco, capaz de:

- 1 Capturar sinais vitais simulados (batimentos cardíacos, temperatura, umidade e movimento).
- 2 Armazenar localmente e garantir resiliência offline (Edge Computing).
- 3 Transmitir dados para a nuvem via MQTT.
- 4 Visualizar informações em dashboards com alertas automáticos.
- 5 Refletir sobre eficiência, segurança e boas práticas em IoT médico.

Observação: O uso de hardware físico é opcional. Caso deseje utilizar equipamentos da FIAP, entre em contato com a tutoria para agendar o laboratório maker.

Parte 1 — Armazenamento e processamento local (Edge Computing)

Desenvolver no Wokwi uma aplicação com ESP32, atendendo aos seguintes requisitos:

- No minimo 2 sensores distintos — um obrigatorio de temperatura (DHT22, que mede temperatura e umidade) e um segundo sensor de livre escolha do grupo.
- [opcional] Armazenamento local no sistema de arquivos SPIFFS. Atencao: o SPIFFS e volatil no simulador — arquivos sao perdidos ao encerrar a simulacao. Funciona apenas em chips fisicos reais. O Monitor Serial pode ser usado como alternativa de resiliencia offline.
- Simular conectividade Wi-Fi via variavel booleana: quando conectado, enviar dados armazenados via Serial.println e apagar o arquivo local.
- Resiliencia offline: o sistema continua coletando dados sem conexao e sincroniza ao voltar online. Definir estrategia de armazenamento limitado alinhada ao modelo de negocio do projeto.

Entregaveis — Parte 1

Link do projeto no Wokwi	Codigo C++ comentado	Relatorio (min. 1 pagina)
--------------------------	----------------------	---------------------------

Parte 2 — Transmissao para nuvem e visualizacao (Fog/Cloud Computing)

Criar um sistema completo de monitoramento que:

- Envie dados simulados do ESP32 para a nuvem via MQTT (ex.: HiveMQ Cloud).
- Monte um dashboard no Node-RED com grafico de sinal vital em tempo real, medidor (gauge) de parametro relevante e indicador visual de alerta quando valores ultrapassarem limites definidos pelo grupo (ex.: >120 bpm, temperatura >38 degC).
- [opcional] Integrar com Grafana Cloud para visualizacao avancada.

Entregaveis — Parte 2

Codigo do ESP32	Prints / export do dashboard	Relatorio (min. 2 paginas)
-----------------	------------------------------	----------------------------

Critérios de avaliacao

Criterio	Pts
Leitura de sensores (SPIFFS nao computado no simulador; chip real = foi alem)	2
Implementacao de resiliencia offline (Edge Computing)	2
Envio via MQTT e integracao com broker	2
Dashboard funcional e alertas automaticos	2
Documentacao clara (codigo + relatorios)	2
Total	10

Desafios Ir Alem

Ir Alem 1 — Comunicacao automatizada com REST e e-mail

Simular um sistema que consuma e envie dados de sinais vitais via API REST em Python, com logica de deteccao de riscos (taquicardia, febre, ausencia de movimento) e acionamento de envio de e-mail automatizado em caso de alerta.

Entregaveis:

- Codigo Python com cliente REST para envio e recebimento de dados.
- Implementacao da logica de verificacao de risco.
- Simulacao de disparo de e-mail automatizado em caso de alerta.
- Relatorio curto (1–2 paginas) descrevendo o fluxo implementado.

Ir Alem 2 — Inteligencia Artificial em series temporais de saude

Aplicar tecnicas de IA para analise de series temporais de sinais vitais. Comparar um classificador tradicional (ex.: regressao logistica) com uma rede neuromorficasimples (modelo LIF ou FHN), explorando vantagens e limitacoes.

Entregaveis:

- Notebook Python comentado.
- Repositorio GitHub publico com codigo e README.
- Relatorio comparativo (2 paginas) sobre o desempenho dos modelos.
- Video de ate 4 minutos no YouTube como nao listado, linkado no README.

Entendendo o projeto — estilo Feynman

Imagine que voce e um medico do futuro. Em vez de examinar o paciente so quando ele aparece no consultorio, voce tem um assistente minusculo grudado no pulso dele, 24 horas por dia, mandando noticias. Esse assistente e o ESP32 — um chipzinho barato que le sensores, pensa um pouquinho, e fala com o mundo.

O problema fundamental

Todo sistema de monitoramento de saude precisa resolver tres perguntas basicas: O que esta acontecendo? (sensores) — Onde guardar isso? (armazenamento) — Quem precisa saber? (transmissao e visualizacao). A atividade inteira e sobre essas tres perguntas. So isso.

Parte 1 — E se a internet cair?

Aqui esta a grande sacada do Edge Computing: imagine que voce esta monitorando o coracao de alguem e a internet cai. O que acontece com os dados daqueles 20 minutos sem conexao? Se voce jogou fora, tem um problema serio. A solucao e obvia: guarda primeiro, manda depois. O chip continua coletando dados no proprio armazenamento local. Quando a internet volta, ele olha para a pilha de dados guardados e manda tudo. Tem um detalhe importante: o chip tem memoria limitada. Entao voce precisa decidir quantas leituras cabem antes de comecar a perder dados. Essa e uma decisao de negocio, nao de programacao.

Parte 2 — Como mostrar isso para alguem?

Dados presos no chip não salvam ninguém. Eles precisam viajar e aparecer numa tela de forma legível. O caminho é: chip -> MQTT -> nuvem -> Node-RED -> dashboard. O MQTT é um protocolo de comunicação projetado para dispositivos pequenos com conexão instável — como um sistema de correios ultra-leve: o chip publica uma mensagem num tópico e quem quiser saber aquele valor assina aquele tópico e recebe automaticamente. O Node-RED é uma ferramenta visual onde você conecta blocos como se fosse Lego, e no final aparece um painel com gráficos, medidores e alertas.

Os desafios extras — onde fica interessante de verdade

Ir Além 1 pergunta: e se, em vez de um humano olhar o dashboard, um programa ficasse de vigia? Você escreve um código em Python que fica checando os dados e, se detectar taquicardia ou febre, manda um e-mail automaticamente. Sem intervenção humana. Isso é o que acontece em UTIs reais. Ir Além 2 vai mais fundo: e se o sistema não só detectar o problema, mas aprender a prever quando ele vai acontecer? Você compara dois métodos — um modelo estatístico tradicional e uma rede neural inspirada em como neurônios reais disparam. A pergunta honesta: qual deles acerta mais, e qual deles você consegue explicar para o médico? As duas coisas importam.

O que você realmente está aprendendo

No fundo, essa atividade não é sobre ESP32 nem sobre MQTT. É sobre uma ideia muito mais antiga: como você garante que informação crítica chega a quem precisa, na hora certa, mesmo quando as coisas dão errado? Engenheiros de saúde, de aviação, de usinas nucleares — todos perdem o sono com essa mesma pergunta. Você está treinando o músculo certo.

Divisão da equipe por habilidade

A equipe foi dividida em quatro funções complementares. Os membros 1 e 2 precisam se alinhar primeiro — o hardware depende do broker pronto para testar o publish MQTT.

Membro 1

Hardware

ESP32 + sensores no Wokwi

Responsável por toda a camada física e de firmware: montar o circuito no Wokwi, programar o loop de leitura dos sensores em C++, implementar a lógica de resiliência offline e publicar o projeto com link público.

Membro 2

Backend

MQTT + nuvem + alertas

Responsável pela espinha dorsal da comunicação: configurar o broker MQTT na nuvem, definir os tópicos, integrar o código do ESP32 ao broker e, opcionalmente, implementar a API REST em Python com detecção de riscos e envio de e-mail.

Membro 3

Frontend

Node-RED + dashboard + Grafana

Responsavel pelo que aparece na tela: montar o dashboard no Node-RED com grafico de sinal vital, medidor gauge, indicadores visuais de alerta e, opcionalmente, integrar com o Grafana Cloud.

Membro 4

Documentacao

Relatorios + IA (Ir Alem 2)

Responsavel por traduzir o trabalho de todos em texto claro: redigir os dois relatorios obrigatorios, revisar a coerencia geral da entrega e, opcionalmente, implementar o notebook de IA comparativa com video.

Checklists por funcao

Marque cada tarefa conforme for concluindo. Itens marcados como [opcional] sao os desafios Ir Alem.

Membro 1

Hardware

ESP32 + sensores no Wokwi

- ☐ Criar o projeto no Wokwi e configurar o ESP32
- ☐ Conectar e testar o sensor DHT22 (temperatura + umidade)
- ☐ Conectar e testar o segundo sensor escolhido pelo grupo
- ☐ Escrever o loop de leitura periodica em C++, comentado
- ☐ Implementar logica de resiliencia offline via Monitor Serial (fila local + sincronizacao ao voltar online)
- ☐ Publicar o projeto no Wokwi e gerar o link publico

Membro 2

Backend

MQTT + nuvem + alertas

- ☐ Criar conta e configurar broker no HiveMQ Cloud
- ☐ Definir os topicos MQTT (ex.: cardio/temperatura, cardio/bpm)
- ☐ Integrar o codigo do ESP32 ao broker (adicionar publish MQTT)
- ☐ Definir e documentar os limites de alerta com o grupo (ex.: >120 bpm, >38 graus C)
- ☐ [opcional] Criar cliente REST em Python para envio e recebimento de dados
- ☐ [opcional] Implementar logica de risco + disparo automatico de e-mail

Membro 3

Frontend

Node-RED + dashboard + Grafana

- ☐ Instalar e configurar o Node-RED localmente
- ☐ Conectar o Node-RED ao broker MQTT do Membro 2
- ☐ Criar grafico em tempo real de um sinal vital (ex.: batimentos cardiacos)
- ☐ Adicionar medidor (gauge) de parametro secundario (ex.: temperatura)
- ☐ Criar indicador visual de alerta para valores fora dos limites
- ☐ Capturar prints do dashboard funcionando e/ou exportar o fluxo Node-RED

Membro 4

Documentacao

Relatorios + IA (Ir Alem 2)

- ☐ Redigir relatorio da Parte 1: fluxo de funcionamento e logica de resiliencia (min. 1 pagina)
- ☐ Redigir relatorio da Parte 2: fluxo MQTT e configuracao do dashboard (min. 2 paginas)
- ☐ Revisar e organizar todos os entregaveis do grupo antes da submissao
- ☐ Garantir que o codigo C++ e os prints estejam coerentes com os relatorios
- ☐ [opcional] Criar notebook Python com regressao logistica vs. rede neuromorficaLIF/FHN)
- ☐ [opcional] Criar repositorio GitHub publico com README e codigo organizado
- ☐ [opcional] Gravar video de ate 4 min e publicar no YouTube como nao listado

Integrar Edge, Fog e Cloud Computing em aplicacoes medicas e um dos maiores desafios atuais da IoT. Nesta fase, voce vivenciara o ciclo completo de captura, processamento, transmissao e visualizacao de dados em saude digital.